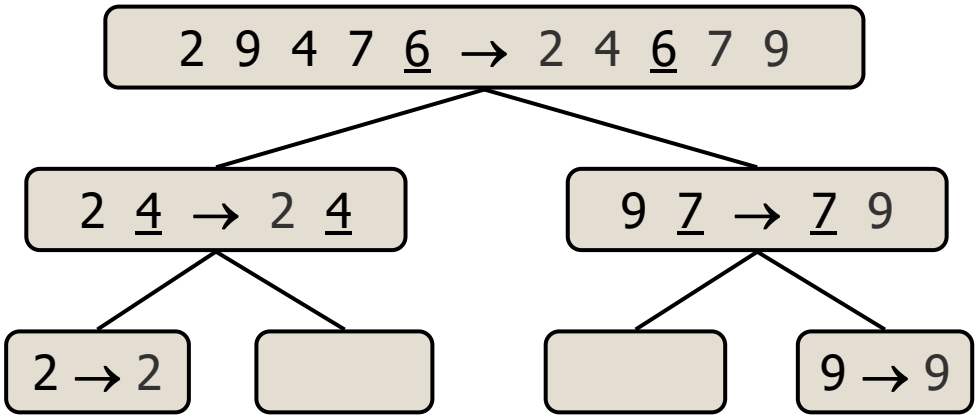
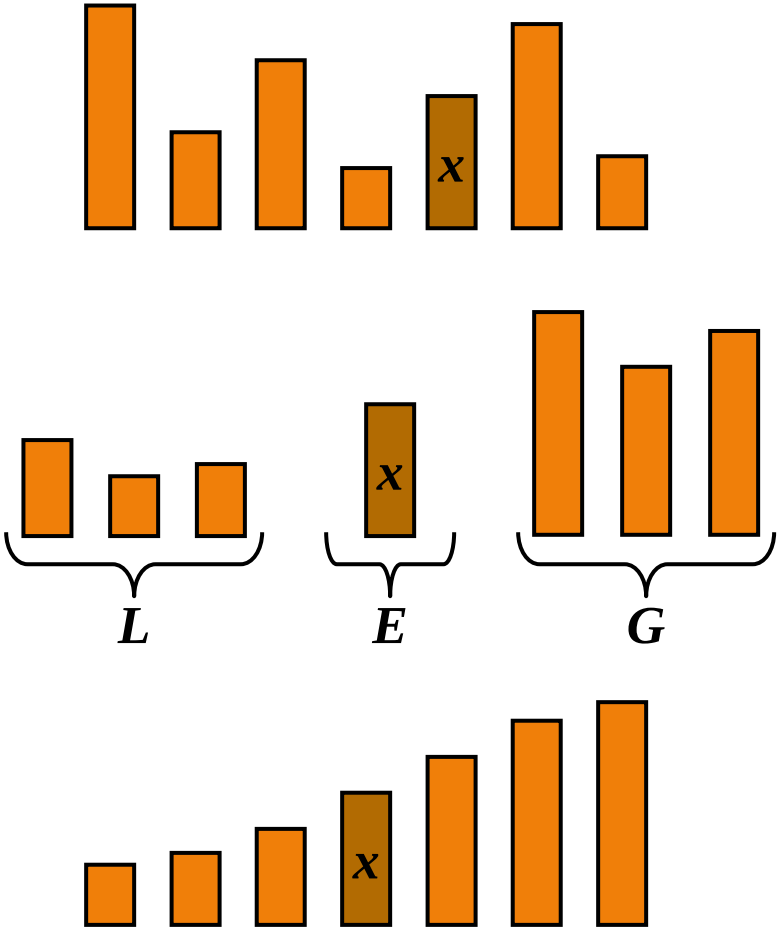
Lecture 5b Quick-Sort



Quick-Sort

- Quick-sort is a sorting algorithm based on the divide-and-conquer paradigm:
  - Divide: pick a element x (called pivot) and partition S into
    - L elements less than x
    - E elements equal x
    - G elements greater than x
  - Recur: sort L and G
  - Conquer: join L, E and G



Quick-Sort (cont.)

- A Quick-Sort algorithm consists of 2 functions/algorithms:
  - A recursive function called quicksort
  - A non-recursive function called partition
- quicksort calls partition

quicksort Function

```
Algorithm quickSort (S, left, right)
  Input: Array S, left index, right index.
  Output: S is sorted.

  if left < right
    pi = partition (S, left, right) // pivot index
    quickSort (S, left, pi-1)       // L, E
    quickSort (S, pi+1, right)      // G
```

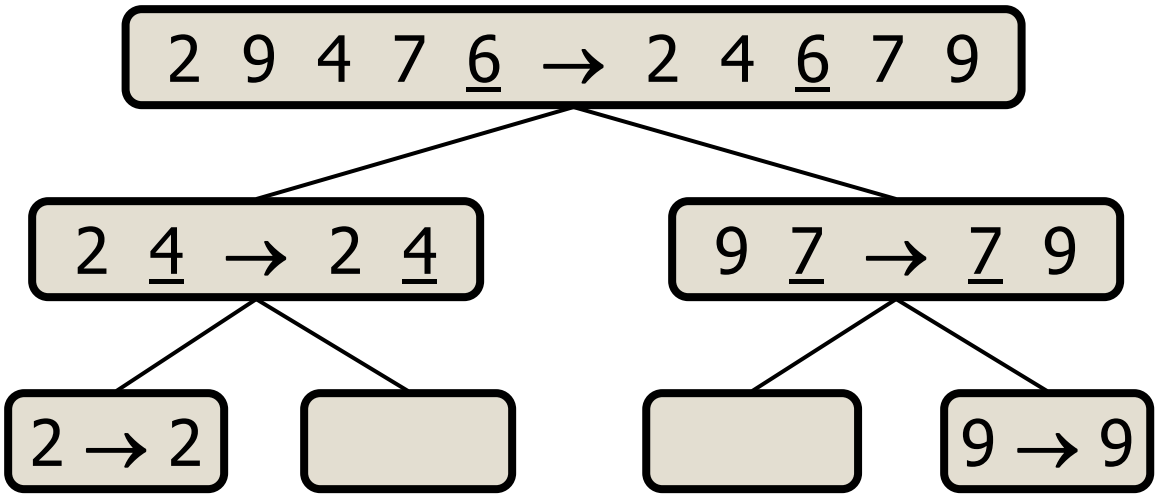
Partition

- We partition an input sequence as follows:
  - We remove, in turn, each element e from S and
  - We insert e into L, E or G, depending on the result of the comparison with the pivot p
  - We remove all elements from L, E and G, and insert it into S
- Each insertion and removal is at the beginning or at the end of a sequence, and hence takes $O(1)$ time
- Thus, the partition step of quick-sort takes $O(n)$ time

```
Algorithm partition(S)
Input: Array S.
Output: All elements < pivot are placed
before pivot, and all elements > pivot
are placed after pivot.

  L, E, G = empty sequences
  p = S.last() // pivot
  while not S.isEmpty()
    e = S.removeFirst()
    if e < p
      L.insertLast(e)
    else if e = p
      E.insertLast(e)
    else // e > p
      G.insertLast(e)

  while not L.isEmpty()
    S.insertLast(L.removeFirst())
  pi = S.size() // pivot index
  while not E.isEmpty()
    S.insertLast(E.removeFirst())
  while not G.isEmpty()
    S.insertLast(G.removeFirst())

  return pi
```

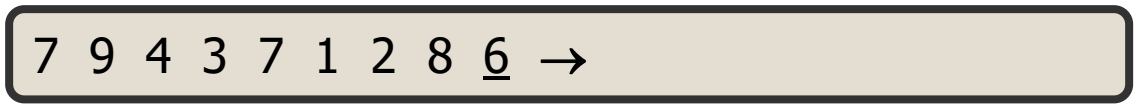
Quick-Sort Tree

- An execution of quick-sort is depicted by a binary tree
  - Each node represents a recursive call of quick-sort and stores
    - Unsorted sequence before the execution and its pivot
    - Sorted sequence at the end of the execution
  - The root is the initial call
  - The leaves are calls on subsequences of size 0 or 1



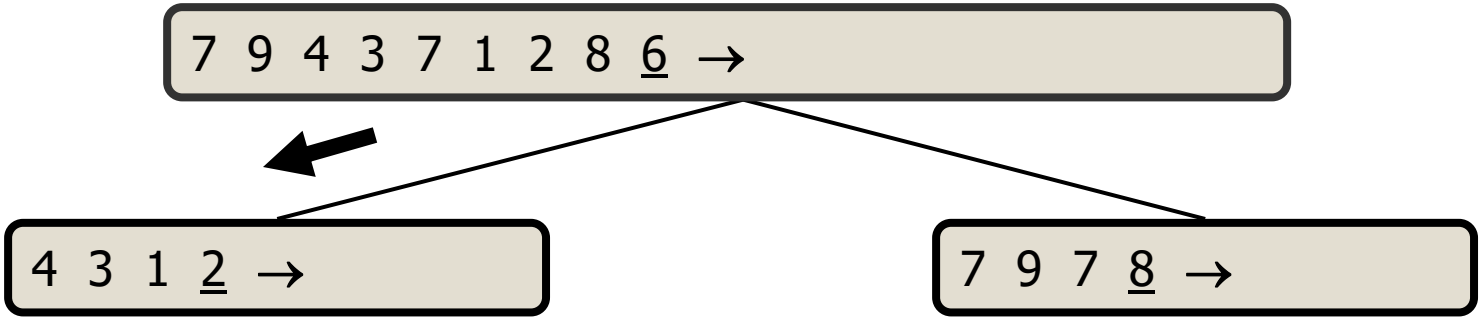
Quick-Sort Tree Example

- Last element as pivot



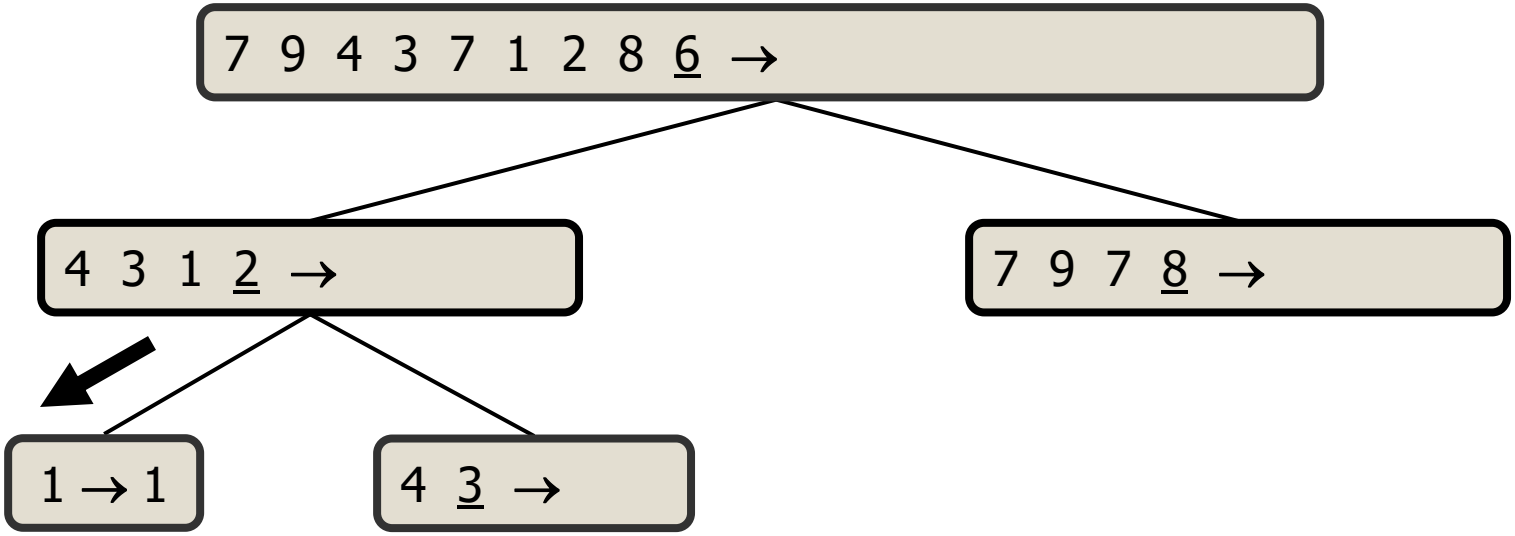
Quick-Sort Tree Example (cont.)

- Partition, recursive call, pivot selection



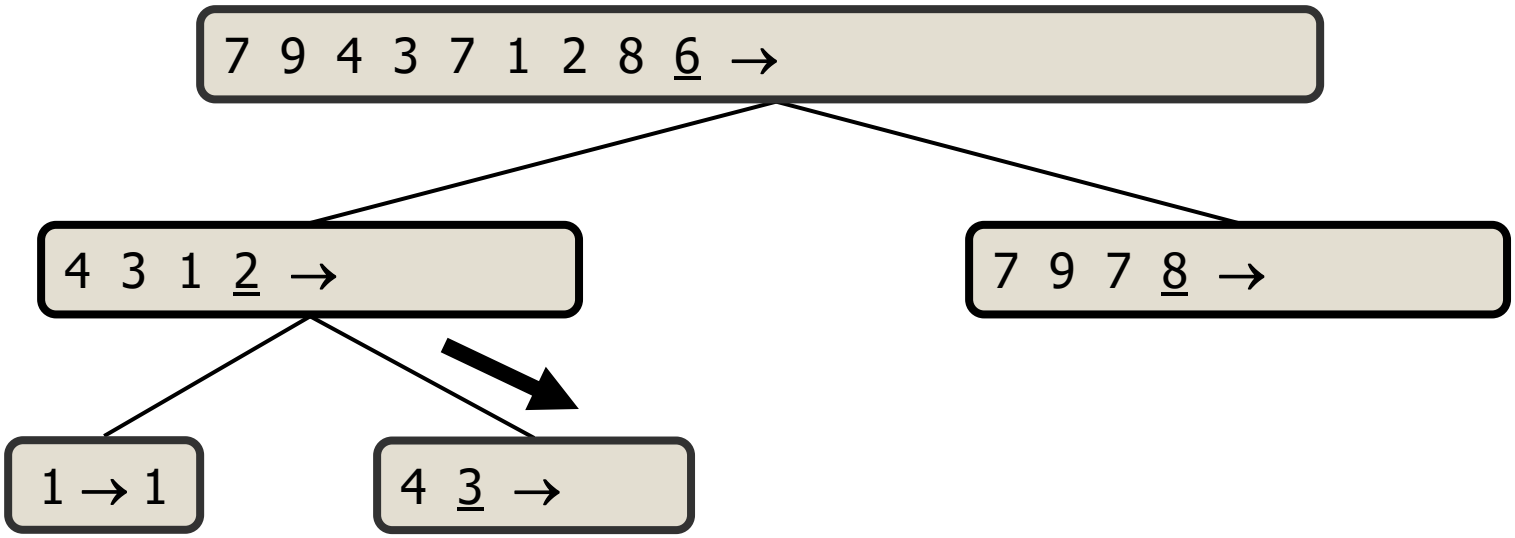
Quick-Sort Tree Example (cont.)

- Partition, recursive call, base case



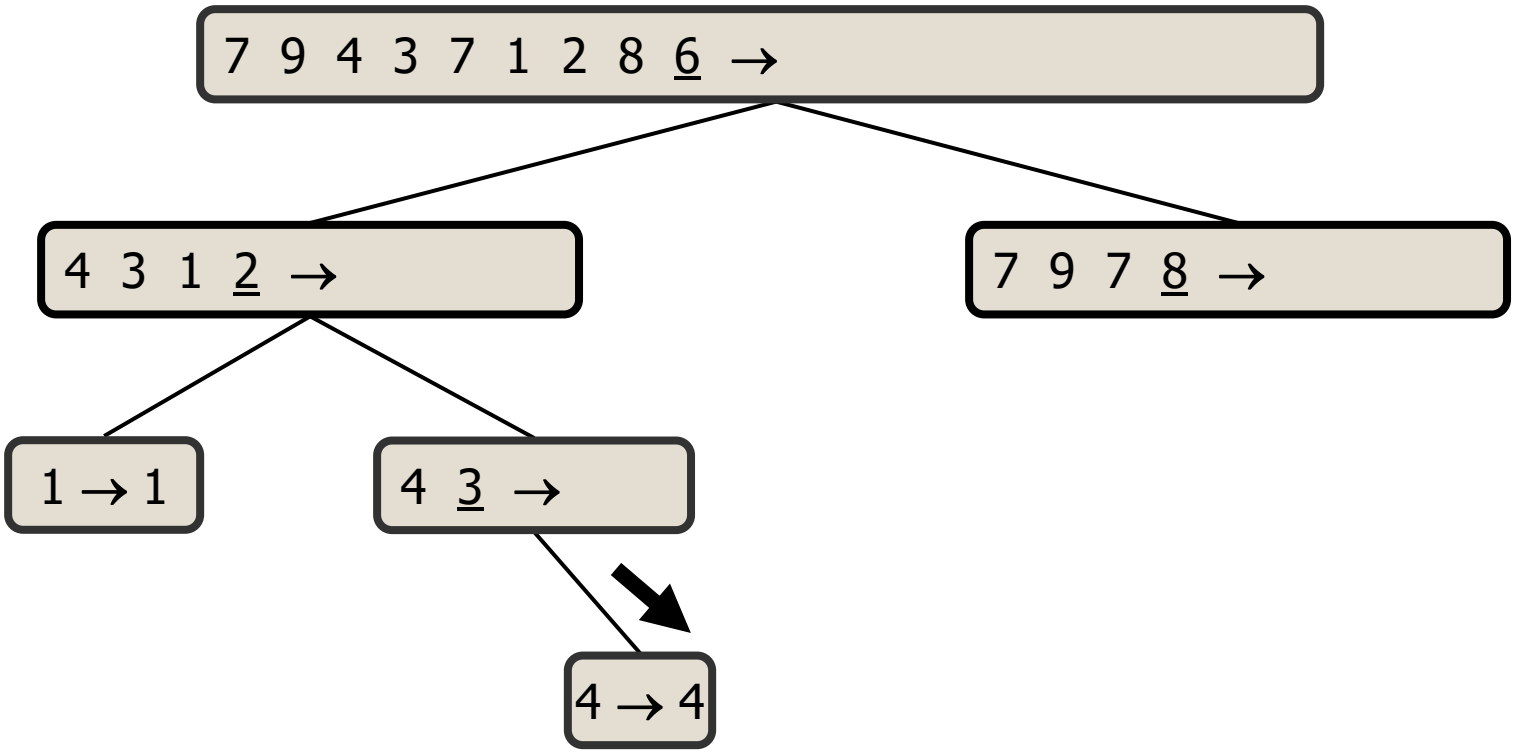
Quick-Sort Tree Example (cont.)

- Recursive call, pivot selection



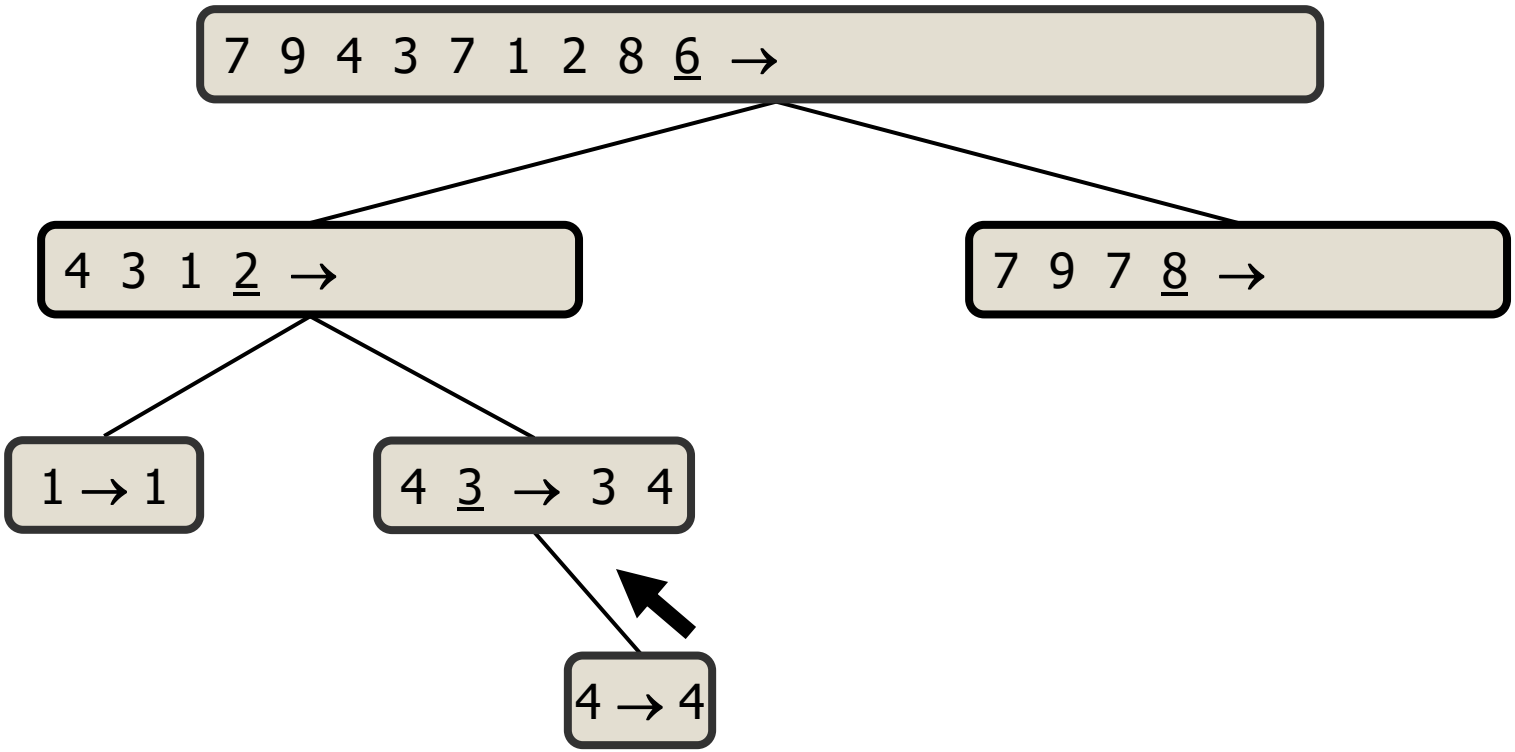
Quick-Sort Tree Example (cont.)

- Partition, recursive call, base case



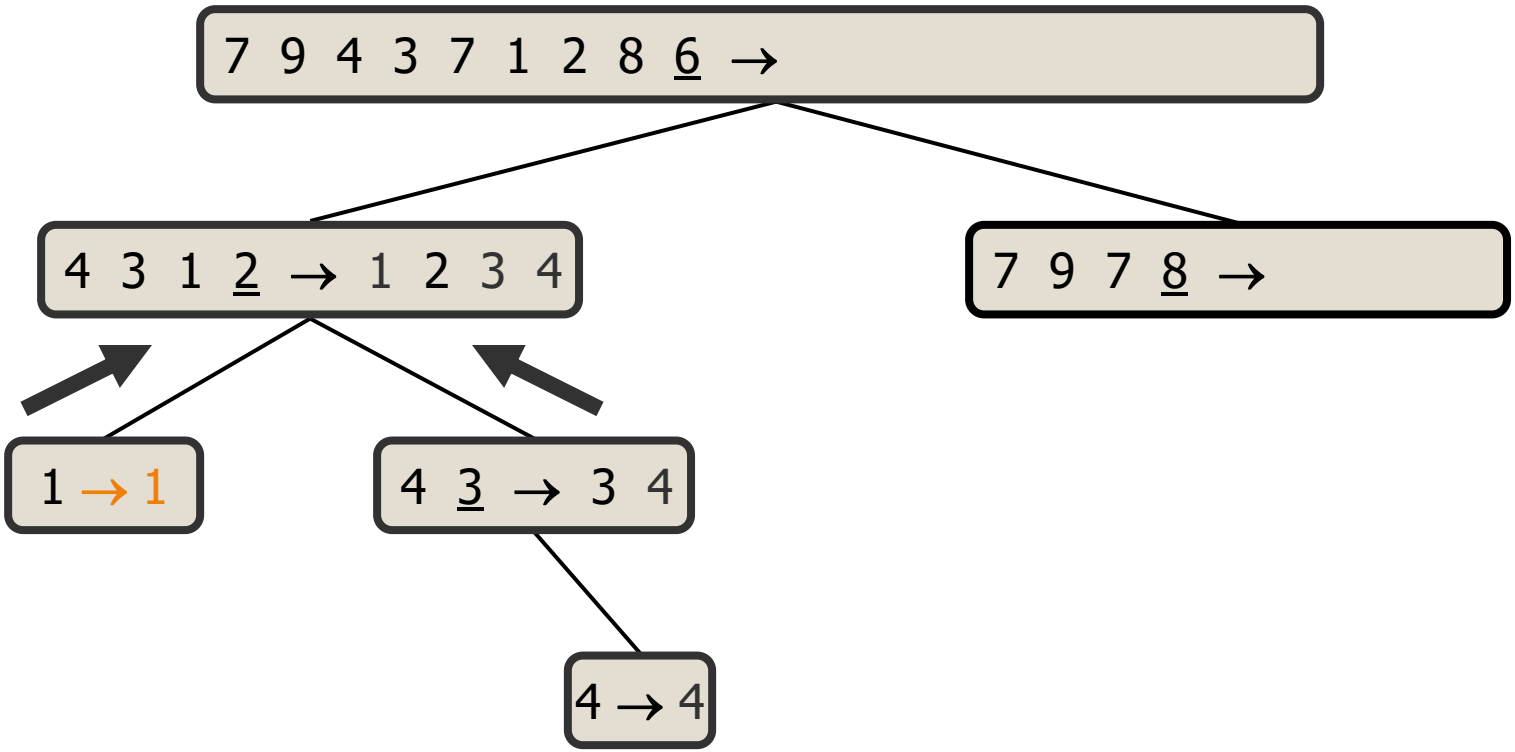
Quick-Sort Tree Example (cont.)

- Join



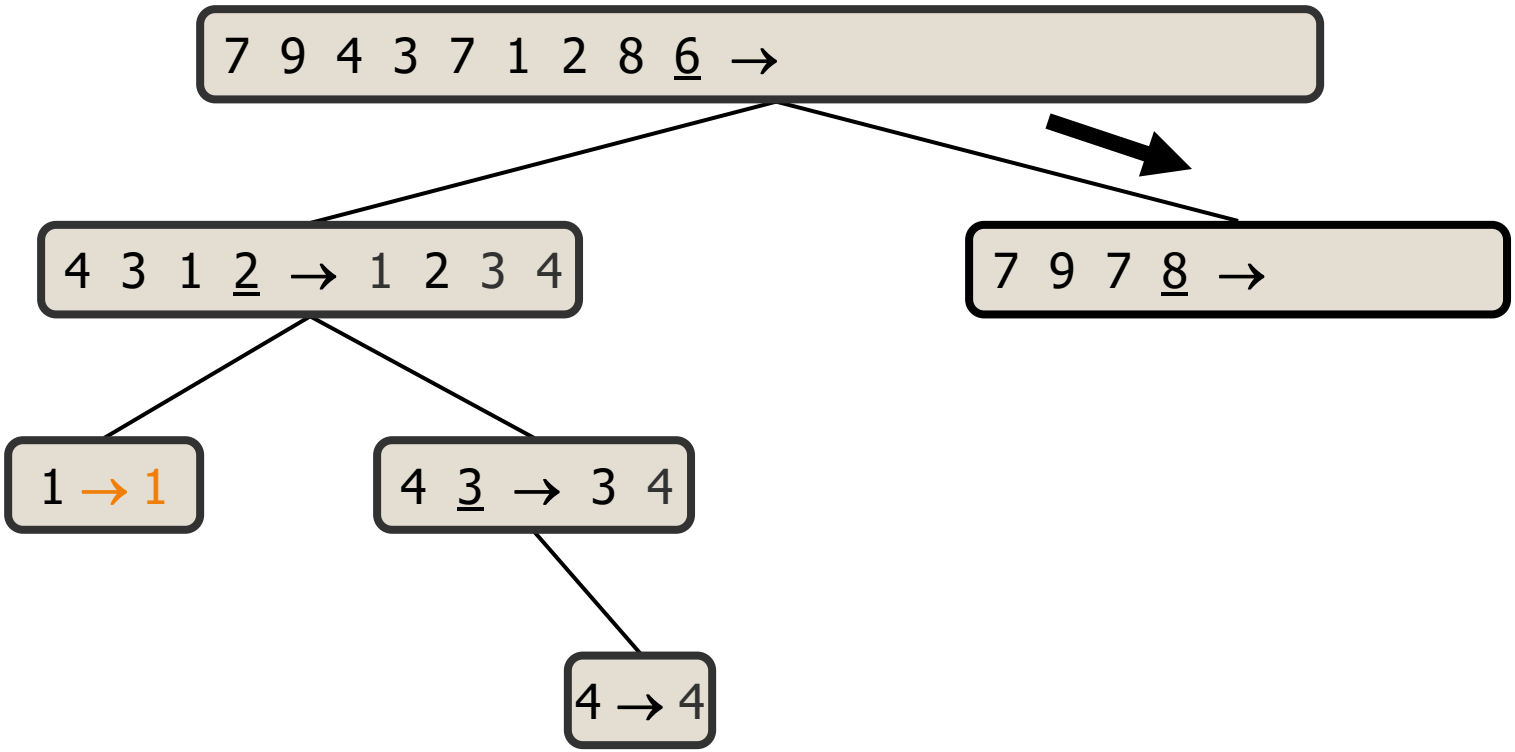
Quick-Sort Tree Example (cont.)

- Join



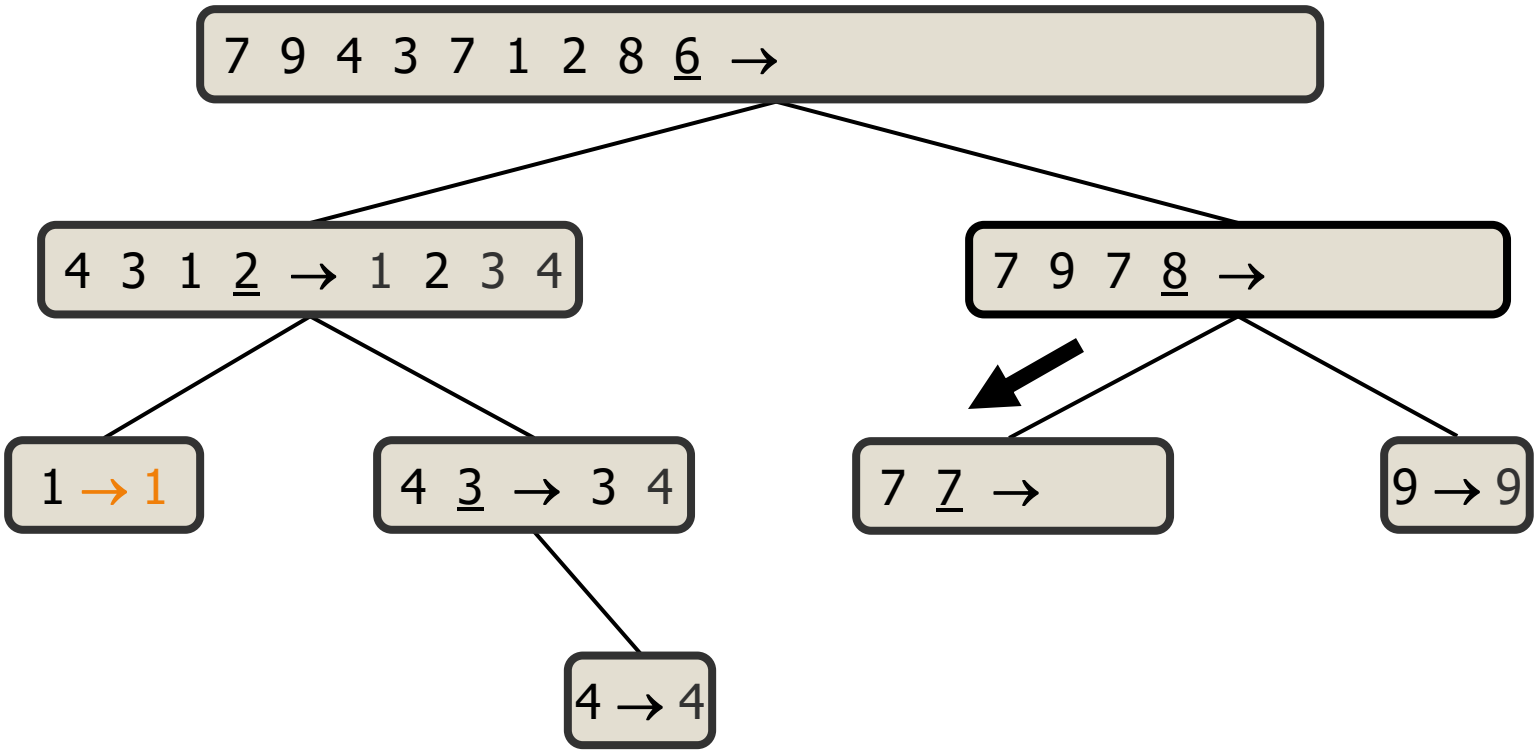
Quick-Sort Tree Example (cont.)

- Recursive call, pivot selection



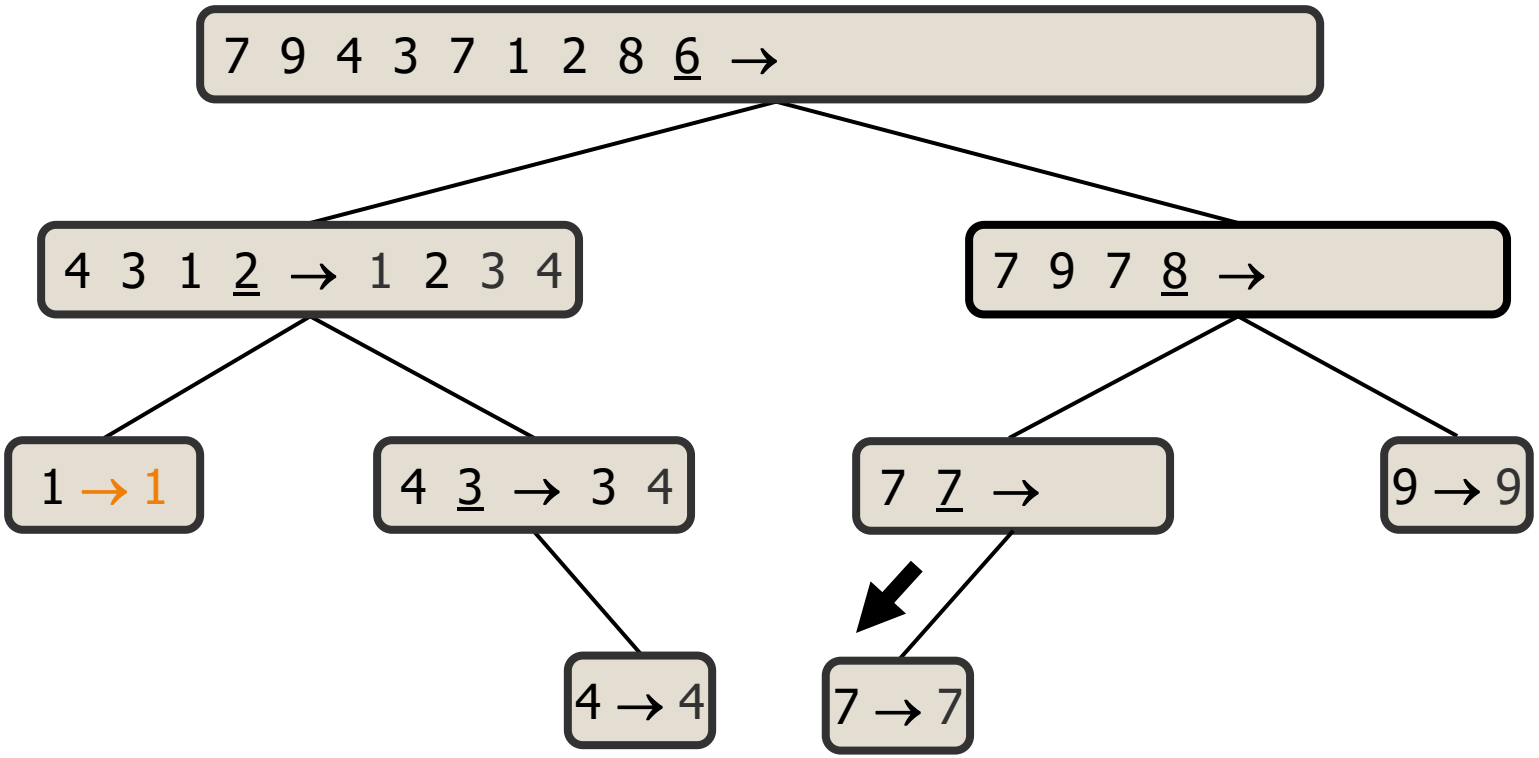
Quick-Sort Tree Example (cont.)

- Partition, recursive, pivot selection



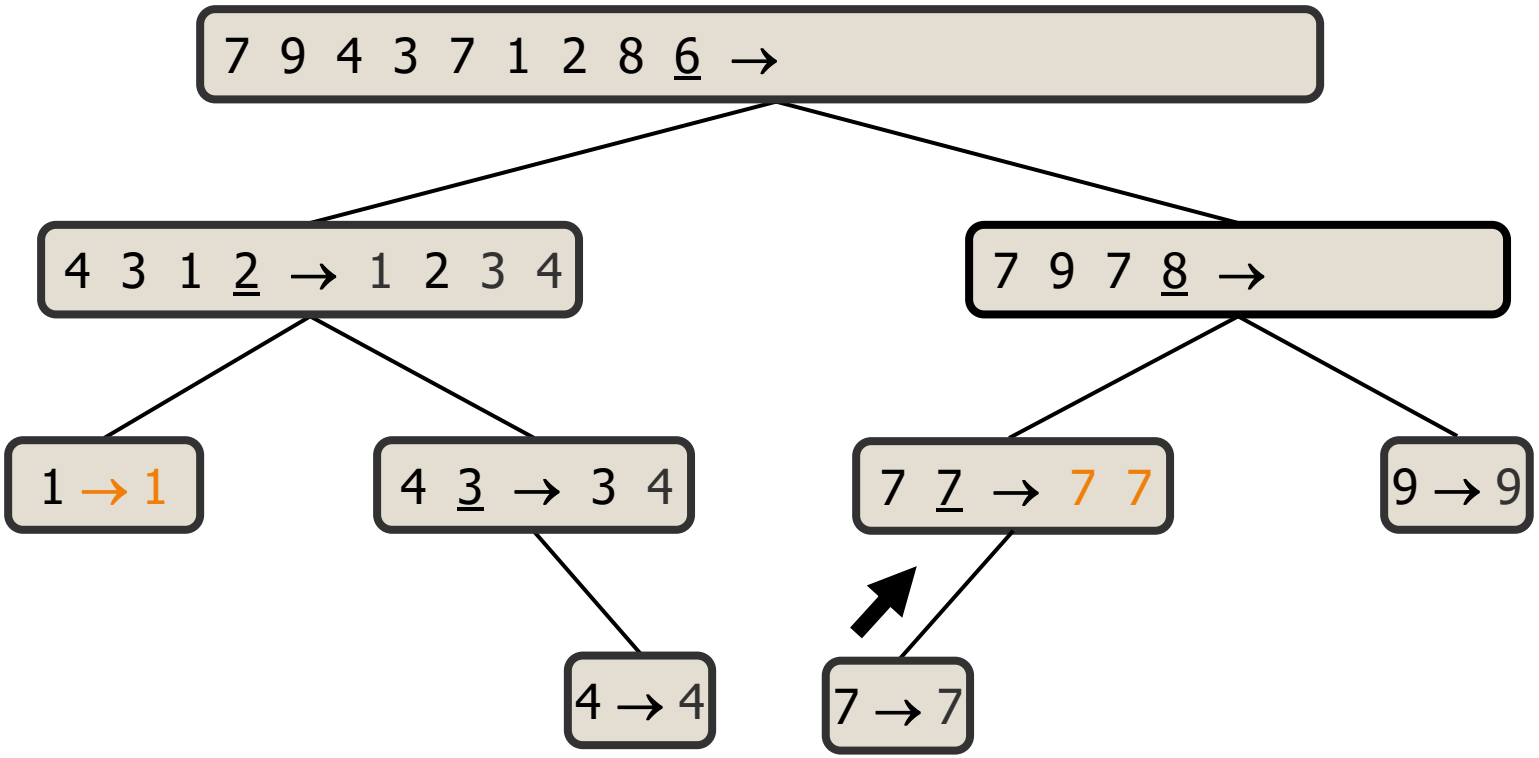
Quick-Sort Tree Example (cont.)

- Partition, recursive, base



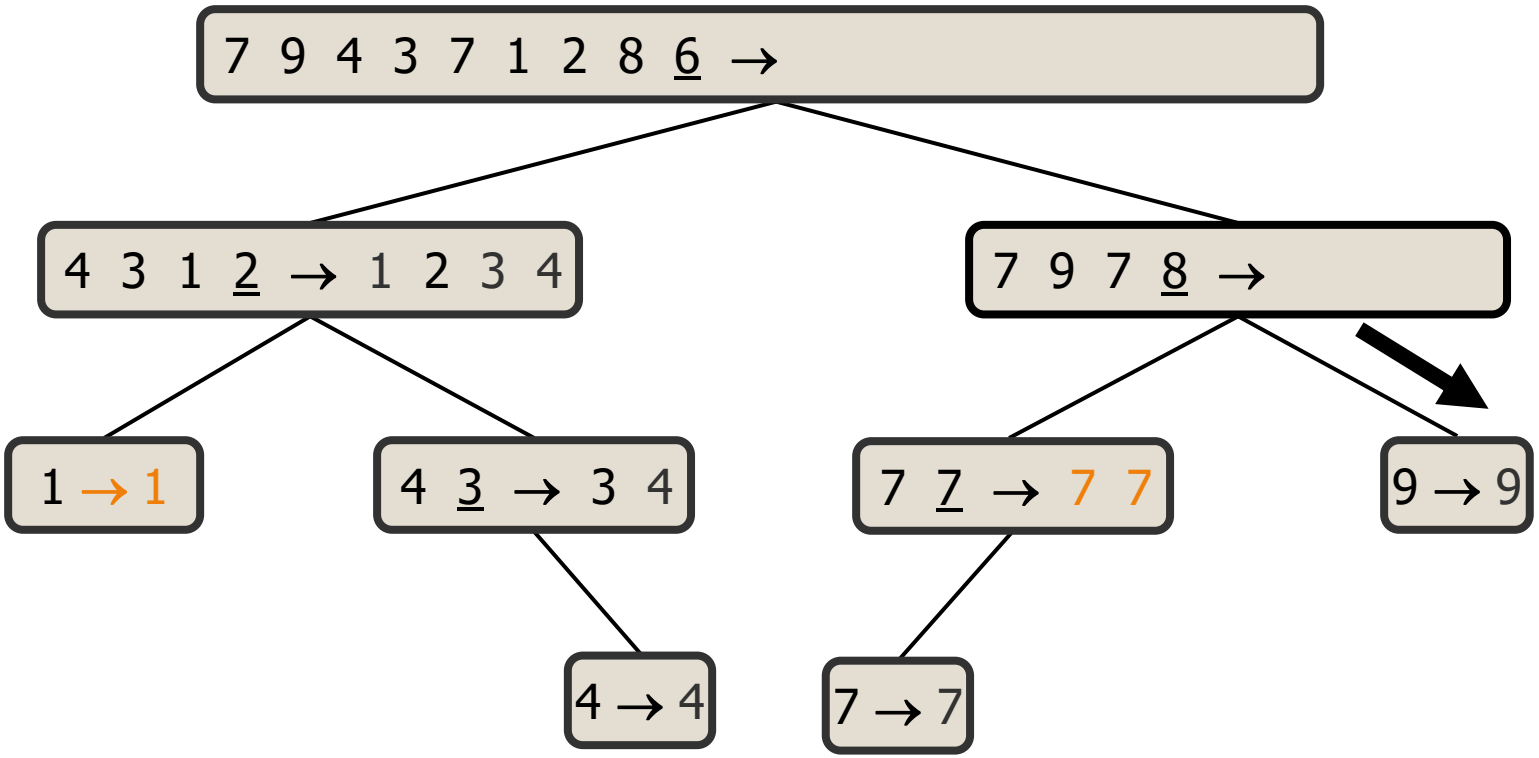
Quick-Sort Tree Example (cont.)

- Join



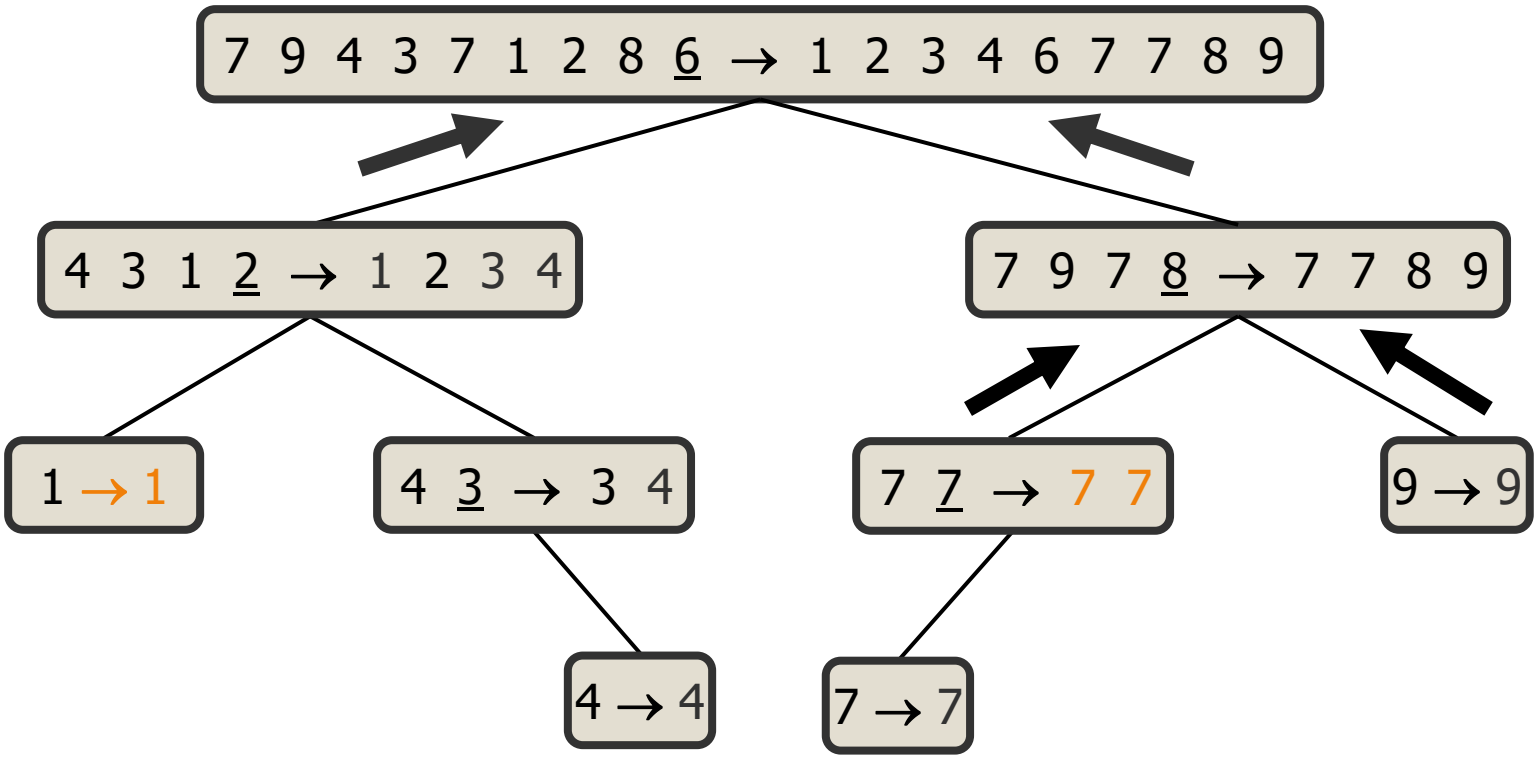
Quick-Sort Tree Example (cont.)

- Recursive, base



Quick-Sort Tree Example (cont.)

- Join, join



Array Output

- The previous Quick-Sort tree and the following array intermediate result are equivalent.

1. 7 9 4 3 7 1 2 8 6
2. 4 3 1 2 6* 7 9 7 8
3. 1 2* 4 3 6* 7 9 7 8
4. 1 2* 3* 4 6* 7 9 7 8
5. 1 2* 3* 4 6* 7 7 8* 9
6. 1 2* 3* 4 6* 7 7* 8* 9

Time Complexities

- Best: $O(n \log n)$ (Proof in Lecture 8).
- Average: $O(n \log n)$
- Worst: $O(n^2)$ if pivot is not randomized, $O(n \log n)$ if pivot is randomized. (Proof in Lecture 8).
- The average case usually dominates over the worst case in sorting algorithms, and in Quicksort, the coefficient of the $n \log n$ term is small, makes Quicksort one of the most popular general-purpose sorting algorithms.

Stability of Sorting Algorithms

- A sorting algorithm is stable if it preserves the order of duplicate keys.
- Array: (2,b), (2,a), (1,a), (3,b)
- (2,b) and (2,a) have the same key value 2.
- (2,b) appears before (2,a) in the original order.
- If during or at the end of sorting, (2,b) always appears before (2,a) then the algorithm is stable. Otherwise, it is considered unstable.

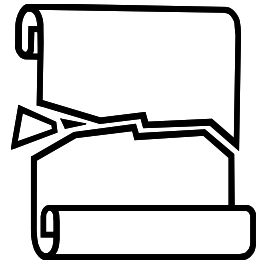
Stability of Sorting Algorithms

- Sequence: (2,b), (2,a), (1,a), (3,b)
- Selection-Sort is unstable
  - Result: (1,a), (2,a), (2,b), (3,b)
- Heap-Sort is unstable
  - Result after 1 dequeue: (2,a), (2,b), (1,a), (3,b)

Stability of Quick-Sort

- The partition algorithm shown earlier is stable and easy to understand. However, it is not space efficient because it requires auxiliary storages to store *L*, *E*, and *G*.
- Space efficient implementations of Quick-Sort are unstable.

Unstable partition Function



- A more efficient but unstable array-based partition of quick-sort takes $O(n)$ time, too.

```
// Lomuto's partition
Algorithm partition (S, left, right)
Input: Array S, left index, right index,
       last element as pivot.
Output: pivot index i+1,
        all elements <= to pivot placed before pivot,
        all elements > pivot placed after pivot.

  i = left - 1
  for j = left to right - 1
    if S[j] <= S[right]
      i++
      swap (S[i], S[j])
  swap (S[i+1], S[right])
  return i + 1
```

Unstable Quick-Sort Output

- If we follow the unstable partition algorithm given earlier strictly, the intermediate results would "seem" more unpredictable as follow:

1. 7 9 4 3 7 1 2 8 6
2. 4 3 1 2 6* 7 9 8 7
   (not 7 9 7 8 anymore)
3. 1 2* 4 3 6* 7 9 8 7
4. 1 2* 3* 4 6* 7 9 8 7
5. 1 2* 3* 4 6* 7 7* 8 9
6. 1 2* 3* 4 6* 7 7* 8 9*